

When Can I Get a Quick Shave?

Alessandro Zanarini^{1,2} and Gilles Pesant^{1,2}

¹ École Polytechnique de Montréal, Montreal, Canada

² CIRRELT, Université de Montréal, Montreal, Canada

{Alessandro.Zanarini,Gilles.Pesant}@cirreлт.ca

Abstract. When designing a constraint model, a crucial underlying question is which inference algorithms to use on the constraints present in that model. Singleton Arc Consistency (SAC) is an effective technique to reduce the search space but it introduces a significant overhead that sometimes outweighs its benefits. Quick Shaving (QS) is a form of reduced SAC that limits this overhead while keeping part of the efficiency. But even QS does not always pay off during search depending on the problem or the instance to which it is applied. In this paper, we study the behavior of QS and propose a coarse grain feature in order to classify problem instances so as to estimate whether or not they may benefit from QS. An empirical study of nine different types of problems and six different search heuristics shows a very similar pattern that can be successfully exploited to dynamically switch off shaving during search. The resulting simple technique automatically determines, with very little overhead, a dosage of QS which is beneficial.

1 Introduction

In Constraint Programming (CP) as with several other computational approaches, good models are key to successfully solving combinatorial problems. But modeling is not the only factor: for example the search strategy can have a tremendous influence on the performance of the approach, so much so that some researchers consider it part of modeling because both factors closely interact. There is still another very important factor that is particular to CP: the level of consistency used for each constraint of the model. Given a model, many combinations of inference algorithms can be applied, often with vastly different performance. Any effort to automate the selection of consistency levels or to dynamically adjust them during search is that much burden being lifted from users of this technology (see [21]).

Strong forms of consistency have proven to be a successful element in tackling hard combinatorial problems with CP. Some of the focus has moved recently from local consistencies to more powerful singleton consistencies. Singleton consistency ensures that a variable-value assignment does not lead to an immediate failure after the constraint network has been propagated. Although singleton consistencies undoubtedly reduce the search space, it is still not univocally accepted whether this reduction outweighs the computational overhead. What makes singleton consistency particularly heavy is the temporary assignment (probe) of

a variable to a value and the following propagation over the whole constraint network just to verify if it is singleton consistent. Furthermore this additional overhead is not always rewarded by some search space reduction but only when the temporary assignment leads to failure.

Singleton consistency has been applied originally to scheduling [14] (under the name of shaving) and to continuous CSPs [12] and was later on formalized by Bessière et al. in [6]. Their solution (Singleton Arc Consistency – SAC) enforces singleton consistency at every node of the search tree on each variable-value pair while keeping the underlying constraint arc consistent; the procedure is iterated until no more pairs can be filtered. Prosser et al. in [17] extended this idea and proposed singleton generalized arc consistency (SGAC) in which they enforce global arc consistency on the constraint network. In order to overcome the significant overhead they also proposed a reduced form of singleton consistency in which, at a given node, each variable-value pair is checked only once. Barták et al. improved SAC in [1] by introducing some sort of support bookkeeping that avoids unnecessary work; the worst-case time complexity remains the same as SAC but experimental results show that their solution outperforms the original SAC. Although Bessière et al. in [2] proposed an algorithm (SAC-Opt) with an optimal time complexity, singleton consistency techniques cannot be blindly applied to any problem. Worth mentioning is also Lecoutre et al. [11] in which singleton consistency is checked through probes that may involve more than one variable-value pair at once.

In order to further lighten the overhead of singleton consistency, some other approaches with a reduced inference power have been developed. Particularly, Lhomme proposed in [13] a reduced form of singleton consistency called Quick Shaving (QS) that, instead of working proactively at each node of the search tree, verifies reactively only variable-value pairs that have recently caused a failure. QS has a theoretically proven small overhead but still not all the problems can benefit from applying such a technique. Ideally all the added probing overhead of QS should be rewarded by a significant pruning of the search space. This depends mainly on two factors: firstly how often the probing detects that a value should be filtered, secondly the impact of the propagation induced by filtering a value through QS (i.e. how much the search space is reduced after shaving a value). To date there is no clear picture of whether or not QS can improve the solving time on a given problem class or instance; furthermore, this possible performance gain also depends on the search heuristics employed during the search i.e. there are problem classes for which some heuristic can improve with QS enabled and some other not. This study tries to shed some light on these issues. Particularly, we propose a first coarse grain feature that could help in the choice of enabling QS during search. This study is not intended to give a definitive answer but rather it is a first step in the direction of understanding the behavior of QS.

The rest of this paper is organized as follows. Section 2 reviews the quick shaving algorithm. Section 3 is dedicated to an experimental analysis of QS. In Section 4 we propose a simple method that improves the behavior of QS. Concluding remarks are then made.

2 Quick Shaving

We briefly introduce some concepts that will be used throughout the paper.

Definition 1. *Given a problem P , a value j of a variable x_i is said to be shavable if θ -consistency³ is able to detect inconsistency on problem $P' = P \cup \{x_i = j\}$. We call shaving attempt a probe on an assignment $x_i = j$ to test whether it is shavable or not.*

Definition 2. *The shaving ratio is defined as the number of shaving attempts that actually prove a value to be shavable for a certain variable over the total number of shaving attempts.*

The basic idea of quick shaving [13] is that each variable-value pair that is detected to be shavable in a branch at depth $k + 1$ should be checked for shaving at depth k . How can we detect values that are shavable at level $k + 1$? From failures, that is if an assignment leads to a failure (a domain wipeout) then the same assignment should be checked for shaving higher up in the search tree. Algorithm 1 shows the pseudo-code of a modified search procedure that integrates QS. The input parameters of the procedure are the current problem P and a branching decision ct ; *shavableList* is an output parameter. Initially the algorithm acknowledges the branching decision (lines 1-2): if a failure is detected then the search backtracks and we annotate the decision that leads to failure (line 4). Otherwise we proceed with a binary search tree trying to assign the unbound variables: the algorithm selects the next branching point and creates an OR node with the decision and its negation (respectively lines 8 and 13). If the left branch succeeds then a solution has been found and the search returns true; if it fails then on the backtrack we trigger the shaving procedure - line 12. The shaving procedure (see Algorithm 2) probes all the variable-value pairs collected in the left branch and it filters those that are actually shavable (those that are not are removed from the shaving list). Note that in case a branching decision and its negation are shavable then the Shave algorithm returns right away a failure. In such a case, Algorithm 1 backtracks and propagates up in the search tree the shaving list (lines 19 to 21). Otherwise, it creates the right branch (line 13); in case the right branch does not succeed the shaving information collected from the left and right branches are propagated up in the search tree.

This is the basic algorithm but, as shown in Example 1, some minor optimizations are possible.

Example 1. Figure 1 shows an example of quick shaving. Nodes are labeled in order of their exploration. The first node that causes a failure is node 3 and the shaving attempt $x_i = j$ is propagated up to node 2 for an eventual shaving. Here we see the first optimization: it is superfluous to probe for shaving $x_i = j$, since the search is about to branch on $x_i \neq j$. The pair $x_i = j$ will be kept in the shaving list and propagated up to node 1 for a shaving attempt.

³ any form of consistency

Algorithm 1: QS-Search(in P, in ct, out shavableList)

```

1 add ct to P;
2 ret  $\leftarrow \theta$ -consistent(P  $\cup$  ct);
3 if ret = failure then
4   shavableList  $\leftarrow$  {ct};
5   return false;
6 if some variables are unassigned then
7   dec  $\leftarrow$  select a branching decision;
8   retLeft  $\leftarrow$  QS-Search(P,dec, shavableLeft);
9   if retLeft = true then
10    return true;
11  else
12    if Shave(P,shavableLeft) then
13      retRight  $\leftarrow$  QS-Search(P,not(dec), shavableRight);
14      if retRight = true then
15        return true;
16      else
17        shavableList  $\leftarrow$  shavableLeft  $\cup$  shavableRight;
18        return false;
19    else
20      shavableList  $\leftarrow$  shavableLeft;
21    return false;
22 else
23   return true;

```

Algorithm 2: Shave(inout P, inout shavableList)

```

1 foreach dec  $\in$  shavableList do
2   ret  $\leftarrow \theta$ -consistent(P  $\cup$  dec);
3   if ret = failure then
4     P  $\leftarrow$  P  $\cup$  not(dec);
5     retNeg  $\leftarrow \theta$ -consistent(P);
6     if retNeg = failure then
7       return false;
8   else
9     shavableList  $\leftarrow$  shavableList  $\setminus$  {dec} ;
10 return true;

```

The right child of node 2 is then created. Both children of node 4 (nodes 5 and 6) bring a failure, therefore node 4 ends up with a shaving list $SL_4 = \{(x_{i'} = j'), (x_{i'} \neq j')\}$. This shaving list will then be propagated up to node 2 whose final shaving list will be $SL_2 = \{(x_i = j), (x_{i'} = j'), (x_{i'} \neq j')\}$. After backtracking, at node 1, shaving probing is finally triggered, attempting to shave the elements of SL_2 . Those that are shavable are kept in the shaving list of node 1 and possibly passed further up in the search tree in case of backtrack; the ones that are not shavable are discarded.

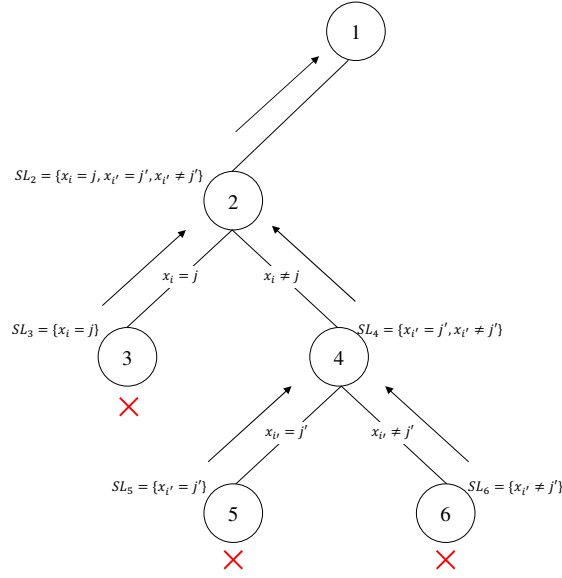


Fig. 1. Quick Shaving Example

3 Understanding Quick Shaving Behaviour

There is undoubtedly a computational price to pay for QS. So in what circumstances should it be applied? We conjecture that a high shaving ratio leads to an actual computational advantage in the search. To put this conjecture to the test, we performed an empirical study over nine different problem types (for a total of about 630 instances) using six different heuristics with and without the quick shaving procedure (for a total of about 640 hours of testing). With one exception, all the heuristics over all the problems showed a very similar behavior: when the shaving ratio is higher than a certain threshold then the total solving time is lower compared to the same heuristic without quick shaving. This study has been conducted with two main aims: firstly, to collect empirical evidence supporting the conjecture; secondly, to determine whether this shaving ratio threshold is an invariant with respect to problem class, instance, and search heuristic.

3.1 Problems

In the following the set of problem classes used for this test is described.

Quasigroup completion problem with holes The Quasigroup completion problem with holes – QWH – (also referred to as Latin Square problem) is defined on a $n \times n$ grid whose squares each contain an integer from 1 to n such that each integer appears exactly once per row and column. The most common model uses

a matrix of integer variables and an **alldifferent** constraint for each row and each column. We tested on the 40 instances used in [22] that have $n = 30$ and 42% of holes and were generated using [10].

Nonograms A Nonogram (problem 12 of CSPLib⁴) is built on a rectangular $n \times m$ grid and requires filling in some of the squares in the unique feasible way according to some clues given on each row and column. Each individual clue indicates how many sequences of consecutive filled-in squares there are in the row (column), with their respective size in order of appearance. We modelled the problem with a matrix of binary variables and using a **regular** constraint for each row and column. The instance set is the same of that [22] and it counts 180 instances.

Multi dimensional knapsack problem This set corresponds to the six mknap instances used in [18] plus 19 more taken by the OR-Library (Weish[1-13] from [19] and PB[1,2,4] and HP[1,2] from [7]). This optimization problem has been transformed into a constraint satisfaction problem by fixing the objective function to its optimal value, thereby introducing a 0-1 equality **knapsack** constraint. The other constraints are upper bounded **knapsack** constraints on the same variables.

Market Split Problem The Market Split Problem consists of m 0-1 equality **knapsack** constraints defined on the same set of $10(m - 1)$ variables. Even small instances ($4 \leq m \leq 6$) are surprisingly hard to solve by standard means. We used the 10 instances tested in [16].

Magic Square Completion Problem This puzzle is defined on a $n \times n$ grid and asks to fill the square with the number from 1 to n^2 such that each row, each column and each main diagonal sums up to the same value. This problem is modelled with a matrix of integer variables, a single **alldifferent** constraint and a **knapsack** constraint for each row, column and main diagonal. In order to avoid trivial solutions, the problem instances have been partially prefilled (half of the instances have 10% of the variables set and the remaining 50% of the variables set). The 40 instances are taken from [16].

Cost-Constrained Rostering Problem Again, this problem has been borrowed from [16] and the 10 instances as well. It is inspired by a rostering problem where four employees have to accomplished a set of tasks in a 25-days schedule. No employee can perform the same task of another employee on the same day (an **alldifferent** constraint on each day). Moreover, there is an hourly cost for making someone work, which varies both across employees and days. For each employee, the total cost must be equal to a randomly generated value (an equality **knapsack** constraint for each employee). In the following, we refer to this problem also as KPRostering.

⁴ www.csplib.org

Rostering Problem This problem has been introduced for this paper and although inspired by a rostering context as the previous one, it has different characteristics w.r.t. the Cost-Constrained Rostering Problem. The schedule is square (10 employees and 10 days) and in a given day the tasks assigned to the employees have to be pairwise different (an **alldifferent** on each day). The schedule of each employee has to follow some rules defined by a **pattern** constraint. We generated 2 sets of 30 instances each with a 5% of preset variables and respectively 0% and 0.025% of values removed.

Travelling Tournament Problem with Predefined Venues The Travelling Tournament Problem with Predefined Venues (TTPPV) has been recently introduced in [15]. The problem is about finding a single round-robin schedule for a sport league such that each team plays no more than 3 consecutive matches at home or away. Furthermore, the venue of every match is already predefined (that is for each team pair $\langle a, b \rangle$ the venue is already set). We used a set of 20 instances taken from [15] for 20 teams, half of which are balanced (the difference of the number of home matches and away matches is at most 1 for every team) and the remaining unbalanced.

Random CSP We generated 245 instances of random CSP following a random model defined by four parameters $\langle n, d, p_1, p_2 \rangle$ (n variables with domain size of d , choosing $\frac{p_1 n(n-1)}{2}$ binary constraints and $p_2 * d^2$ forbidden tuple in each constraint). We used a generator⁵ that implements a Model B generator (see [9]): n has been set to 150, d to 7 and we varied p_2 from 0.2 to 0.8 with steps of 0.1 and κ from 0.8 to 1.1 with steps of 0.05 (phase transition i.e. hard instances occurs around $\kappa \approx 1$). For each combination of p_2 and κ we generated 5 instances with p_1 computed accordingly to the formula (see [8]):

$$\kappa = \frac{-|C| \log_2(1 - p_2)}{n \log_2(d)}$$

where C is the set of binary constraint.

3.2 Heuristics

The six heuristics we tested are briefly described here:

- Brélaz-Lex ([4]): it selects the set of variables with the smallest domain and then it breaks ties choosing the variable with highest dynamic degree. Value selection is performed in lexicographic order.
- Dom/ddeg-Lex ([3] and [20]): it selects the variable that minimizes the domain cardinality over the dynamic degree⁶. Value selection is performed in lexicographic order.

⁵ <http://www.lirmm.fr/~bessiere/generator.html>

⁶ Also referred to as forward degree or future degree

- Impact Based Search (IBS) [18]: it selects first the variable whose instantiation triggers the largest search space reduction (highest impact) that is approximated as the reduction of the cartesian product of the variables’ domains. Then it selects the value with the smallest impact.
- Maximum Solution Density (maxSD) [22]: it extracts solution counting information from the constraints and then it branches on the variable-value pair with the highest solution density. We use the counting algorithms presented in [22] for the `regular` and `pattern` constraint, in [16] for the `knapsack` constraint; for the `alldifferent` constraint the counting algorithm is unpublished yet.
- Brélaz-maxSD: it selects the variable accordingly with the Brélaz heuristic and then it chooses the value with the maximum solution density.
- Dom/ddeg-maxSD: it selects the variable accordingly with the Dom/ddeg and then it chooses the value with the maximum solution density.

Heuristics based on solution counting assume the presence of some countable constraints in the problem model, therefore the last three heuristics have not been tested on random CSP problems. For all the heuristics, we used binary branching (i.e. $x_i = j \parallel x_i \neq j$).

3.3 Experimental Analysis

Each heuristic has been tested with and without Quick Shaving on all the problem instances described in the previous section (with the exception of random CSPs that have been tested only with Brélaz-Lex, Dom/ddeg-Lex and IBS). Every problem has been modelled such that domain consistency is enforced on each constraint. The experiments were run on a AMD Opteron 2.4 GHz and 1GB of RAM with Ilog Solver 6.6. We set a timeout of 20 minutes for each problem instance. As an indication of the problems’ difficulty, we present in Table 1 the raw results of the heuristics without QS.

To study the performance gain brought by the quick shaving procedure, we define for each heuristic and problem class, the time ratio as the average solving time with QS enabled over the average solving time with QS disabled; a time ratio lower than one indicates on average a performance gain of the heuristic with QS enabled, whereas a time ratio higher than one means a higher solving time with QS. We plot the time ratios in Figure 2.

The Latin Square, Nonograms, Rostering and TTPPV are the problems that benefit the most from QS independently from the heuristic (except for IBS in Rostering). The Market Split and Magic Square problems show an important overhead for QS. As for the other problems, there is a slight overhead for Multi-Knapsack (from 5% to 26% worse than traditional search with no quick shaving), KPRostering (from 3% to 6% except for MaxSD), Rostering (17% for IBS), and Random CSP (from 2% to 7%). Note that if we consider only instances that are not too easy, the overhead becomes even more significant (as an example, MultiKnapsack, KPRostering, and Random CSP without easy instances have an average overhead of 57%, 50%, and 32% respectively). Performance improvements depend mainly on the problem class, nonetheless they may also vary based

heuristic	LatinSquare			Nonograms			MultiKnapsack		
	time	bckts solved		time	bckts solved		time	bckts solved	
Brélaz - Lexico	750.6	1458276	48%	296.9	193618	79%	497.9	11212	64%
dom/ddeg - Lexico	723.6	1387456	55%	296.4	194889	79%	499.5	11130	64%
IBS	716.3	900253	45%	7.4	4712	99%	354.0	4080	76%
maxSD	105.2	104672	98%	48.5	18096	97%	72.7	1228	96%
Brélaz - MaxSD	322.4	569170	83%	294.8	184267	78%	504.1	9948	64%
dom/ddeg - MaxSD	294.5	526034	85%	295.2	186598	78%	501.0	9877	64%
	MagicSquare			MarketSplit			KPRostering		
	time	bckts solved		time	bckts solved		time	bckts solved	
Brélaz - Lexico	658.7	72877	48%	121.6	109742	100%	679.4	995657	50%
dom/ddeg - Lexico	736.0	83113	40%	122.4	109742	100%	681.4	1004048	50%
IBS	638.5	24447	50%	202.3	90936	100%	510.8	264037	60%
maxSD	12.4	1756	100%	256.9	59878	100%	0.3	5	100%
Brélaz - MaxSD	29.9	10741	100%	245.9	93213	100%	655.0	743228	50%
dom/ddeg - MaxSD	138.2	53561	95%	231.8	93213	100%	657.5	748973	50%
	Rostering			TTPPV			RandomCSP		
	time	bckts solved		time	bckts solved		time	bckts solved	
Brélaz - Lexico	894.4	8694226	27%	600.1	971626	50%	198.7	49308	86%
dom/ddeg - Lexico	894.5	8760347	27%	480.1	799315	60%	164.1	38219	89%
IBS	60.5	76310	97%	373.2	309467	70%	161.1	39828	90%
maxSD	934.5	3132784	23%	540.4	551354	55%			
Brélaz - MaxSD	1121.6	6983655	7%	780.1	1113292	35%			
dom/ddeg - MaxSD	1121.6	6969162	7%	696.7	1004713	45%			

Table 1. Solving time (in seconds), number of backtracks and percentage of solved instances without quick shaving

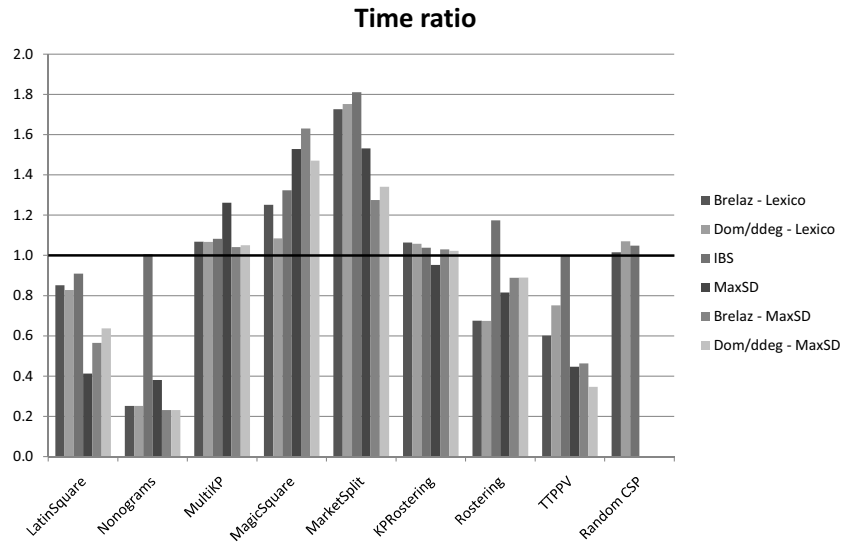


Fig. 2. Time Ratio: average time with QS over average time without QS

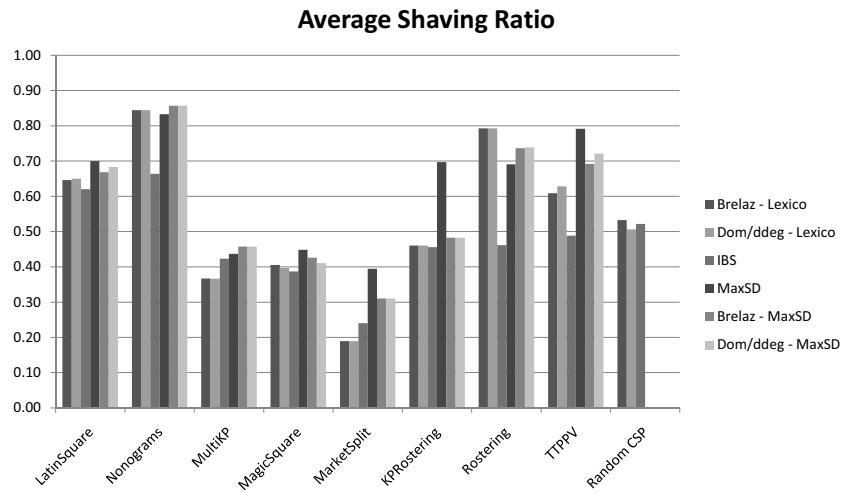


Fig. 3. Average Shaving Ratio

on the heuristics employed: this is the case, for example, of Nonograms problems in which every heuristic took advantage of QS except IBS; the same happened for Rostering where IBS does better without QS whereas all the other heuristics gained from it. Note also that IBS was the only heuristic for which QS brought advantages just in the Latin Square problem.

Figure 3 shows the average shaving ratio for each problem class and heuristic. Problems and heuristics that suffered from QS in terms of solving time are those for which the shaving ratio is low whereas whenever the shaving ratio is sufficiently high we actually have a significant gain. For example in TTPPV, counting based heuristics (maxSD, brelaz-maxSD, domddeg-maxSD) are the ones with higher shaving ratio and this is directly reflected in the time ratio. Note also that the shaving ratio of IBS is in general very low compared to the other heuristics.

What sets IBS apart from the other heuristics in this study is possibly its heavy bias toward the fail-first principle. Its low shaving ratio could be explained by the fact that an inconsistency between two variable assignments is detected early on and thus is not much propagated up the search tree through the shaving lists. This leads to a very low shaving ratio. On the other side heuristics such as maxSD lean more toward succeed-first hence, although it is very competitive, an inconsistency may occur between two variable assignments that are relatively far apart on a branch of the search tree. This means that a variable-value pair that causes a fail can be propagated through the shaving list higher in the search tree hence allowing several times the shaving of that pair.

For all the heuristics, we plotted the time ratio vs the shaving ratio of each individual problem instance (630 in total). From the plot, all the instances that were either too easy or too hard (solved within 2 seconds by the heuristic without QS or unsolved by the heuristic with and without QS) were removed, as well as the instances for which the amount of shaving was too low to be statistically significant i.e. with less than 50 shaving attempts; note, as a reference, that the average number of shaving attempts for the Latin Square problem is of the order of 10^5 . The average number of remaining instances is about 170 (ranging from 120 to about 210).

Surprisingly, all the plots show a similar behavior: most of the points with a time ratio lower than 1 (performance gain with QS) are situated in an area with a shaving ratio higher than ≈ 0.6 . If the shaving ratio goes below this threshold then the time ratio is higher than 1 i.e. QS brings no advantage or even worse it slows down the resolution process. IBS is the only heuristic for which such a split is not as sharp as in the other plots. Figure 5 shows how the instances are clustered by problem class for the dom/ddeg heuristic (note that the other heuristics present a similar clustered pattern): for example, nonograms have a time ratio close to 0 with a corresponding very high shaving ratio; Market Split are situated in the area with the lowest shaving ratio. Although different heuristics may skew a bit the plots and have different results (as shown in Figure 2), the main factor that influences the shaving ratio is the problem class.

To better investigate the relationship between shaving ratio and performance gain, we present in the following the correlation coefficient between the two. We use the phi-correlation coefficient $\phi = \sqrt{\frac{\chi^2}{N}}$ (where χ is the Pearson's chi-square test and N is the population size) to numerically prove the correlation shown in the plots (see [5]). The coefficient varies from -1 to 1 where the extremes mean a perfect anticorrelation and correlation and 0 stands for no correlation. This coefficient assumes that the sample population has two features with two values

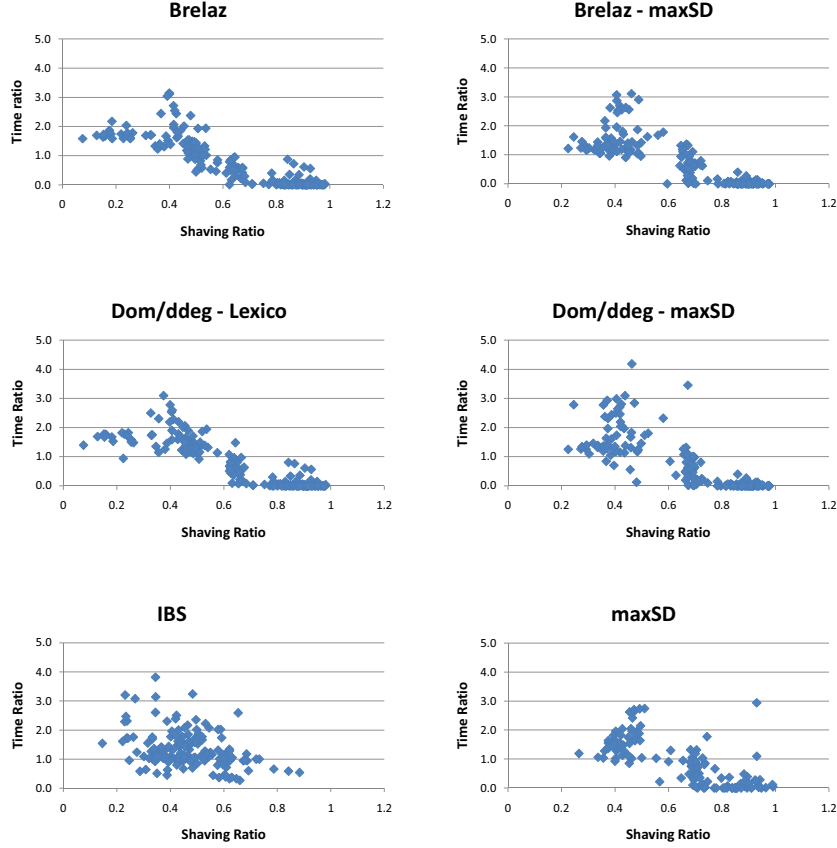


Fig. 4. Scattered plots for the six heuristics showing the Time Ratio vs Shaving Ratio

each. In our context the features are the shaving ratio (higher or lower than a given threshold th) and the time ratio (higher or lower than 1). We found the best correlation with $th \in [55\%, 60\%]$. The correlation for different thresholds are shown in Table 2. Note that this threshold might be slightly dependent on the shaving implementation, however the interesting fact is that no matter which implementation you use there is a sharp threshold above which there is an actual gain in employing QS. For sake of clarity this threshold is valid across the heuristics (with some reservation for IBS) and across the problem classes. All the heuristics show a strong correlation while IBS has just a moderate correlation.

4 Dynamically Disabling Quick Shaving

Ideally a shaving procedure should justify its probing overhead through a significant reduction of the search space that translates to a computational perfor-

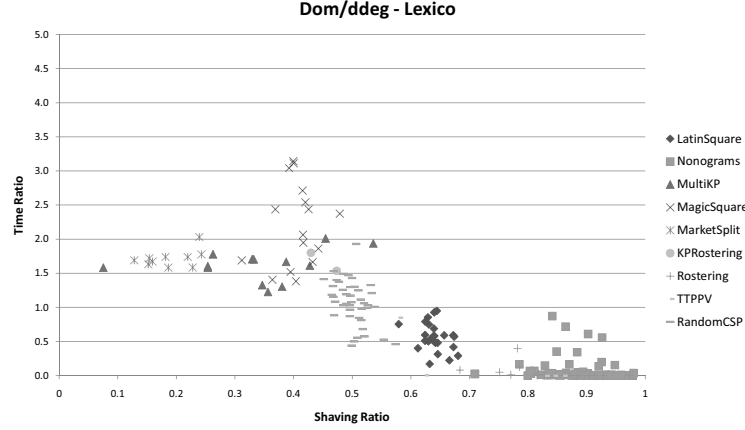


Fig. 5. Scattered plots

Heuristic	$th = 50.0\%$	$th = 52.5\%$	$th = 55.0\%$	$th = 57.5\%$	$th = 60.0\%$
Brélaz - Lexico	0.85	0.83	0.88	0.86	0.85
dom/ddeg - Lexico	0.88	0.92	0.95	0.95	0.96
IBS	0.34	0.44	0.49	0.48	0.49
maxSD	0.81	0.82	0.82	0.82	0.82
Brélaz - MaxSD	0.84	0.85	0.85	0.86	0.86
dom/ddeg - MaxSD	0.81	0.84	0.84	0.84	0.85

Table 2. ϕ correlation coefficient

mance gain. As pointed out earlier, this is not the case for every problem. We can however exploit the behavior of QS — a sharp threshold around a shaving ratio of 55% ~ 60% — to dynamically disable shaving whenever this threshold is not reached. The procedure keeps track of the shaving statistics and after K shaving attempts it starts checking the value of the shaving ratio: if it is below the threshold then QS is switched off and never enabled again. For the experimentation, we set K equal to 100, which is a considerably low number of shaving attempts, and the threshold is set to 57%.

Figure 6 shows the results: recall that although the plots in Figure 4 and the correlation coefficient were done only on a subset of the instances (not too easy and not too hard and with a reasonable amount of quick shaving), here we present the average gain or loss over all 630 instances. QS with the shaving filter adds almost no running time overhead to any of the problem classes / heuristics; in the best case it behaves very similarly to pure QS. Despite just a moderate correlation, IBS took advantage of the shaving filter and was able to improve the results over QS in all the problem classes; nonetheless, the shaving filter was not able to completely clear the overhead for the market split problem but it considerably reduced it passing from a time ratio of 1.80 to 1.15.

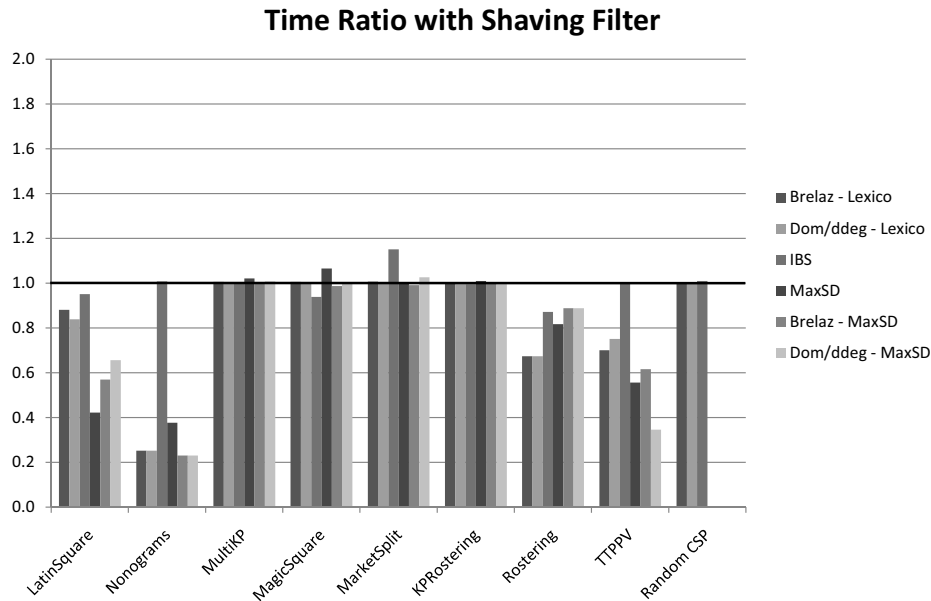


Fig. 6. Time Ratio: average time of QS with shaving filter over average time without QS

5 Conclusion

This paper studied the behavior of quick shaving combined with several search heuristics and applied to several problem classes. Out of the empirical data collected, a simple but telling feature emerged, allowing quick shaving to be tried with almost no computational overhead in cases where it shouldn't and with significant computational advantage in cases where it should. The proposed procedure is very easy to implement on top of quick shaving and relieves the user from deciding whether or not to apply shaving. This is by no means a definitive answer in understanding strengths and weaknesses of QS but only a first step: in the future, we would like to better investigate also the relation between the reduction of the search space induced by QS and its performance.

References

1. R. Barták and R. Erben. A New Algorithm for Singleton Arc Consistency. In *Proceedings of FLAIRS04*, 2004.
2. C. Bessière and R. Debruyne. Optimal and Suboptimal Singleton Arc Consistency Algorithms. In *Proceedings of IJCAI'05*, pages 54–59, 2005.
3. C. Bessière and J.-C. Régin. MAC and Combined Heuristics: Two Reasons to Forsake FC (and CBJ?) on Hard Problems. In *Proceedings of CP96*, pages 61–75, 1996.

4. D. Brélaz. New Methods to Color the Vertices of a Graph. *Communications of the ACM*, 22(4):251–256, 1979.
5. P. Y. Chen and P. M. Popovich. *Correlation. Parametric and Nonparametric Measures*. Sage Publications, Inc, 2002.
6. R. Debruyne and C. Bessiere. Some Practicable Filtering Techniques for the Constraint Satisfaction Problem. In *Proceedings of IJCAI97*, pages 412–417, 1997.
7. A. Freville and G. Plateau. A Branch and Bound Method for the Multiconstraint Zero One Knapsack Problem. *Investigation Operativa*, 1:251–270, 1990.
8. I. P. Gent, E. MacIntyre, P. Prosser, B. M. Smith, and T. Walsh. An Empirical Study of Dynamic Variable Ordering Heuristics for the Constraint Satisfaction Problem. In *In Proceedings of CP96*, pages 179–193. Springer, 1996.
9. I. P. Gent, E. Macintyre, P. Prosser, B. M. Smith, and T. Walsh. Random Constraint Satisfaction: Flaws and Structure. *Constraints*, 6(4):345–372, 2001.
10. C. Gomes and D. Shmoys. Completing Quasigroups or Latin Squares: A Structured Graph Coloring Problem. In *COLOR 2002: Proceedings of Computational Symposium on Graph Coloring and Generalizations*, pages 22–39, 2002.
11. C. Lecoutre and S. Cardon. A greedy approach to establish singleton arc consistency. In *Proceedings of the 19th International Joint Conference on Artificial Intelligence (IJCAI'05)*, pages 199–204, Edinburgh, Scotland, August 2005.
12. O. Lhomme. Consistency Techniques for Numeric CSPs. In *Proceedings of IJCAI93*, pages 232–238, 1993.
13. O. Lhomme. Quick shaving. In M. M. Veloso and S. Kambhampati, editors, *AAAI*, pages 411–415. AAAI Press / The MIT Press, 2005.
14. P. Martin and D. B. Shmoys. A New Approach to Computing Optimal Schedules for the Job-Shop Scheduling Problem. In *Proceedings of the 5th International IPCO Conference*, pages 389–403. Springer, 1996.
15. R. Melo, S. Urrutia, and C. Ribeiro. The Traveling Tournament Problem with Predefined Venues. *to appear in Journal of Scheduling*, 2009.
16. G. Pesant and C.-G. Quimper. Counting solutions of knapsack constraints. In L. Perron and M. A. Trick, editors, *CPAIOR*, volume 5015 of *Lecture Notes in Computer Science*, pages 203–217. Springer, 2008.
17. P. Prosser, K. Stergiou, and T. Walsh. Singleton Consistencies. In *Proceedings of CP02*, pages 353–368, London, UK, 2002. Springer-Verlag.
18. P. Refalo. Impact-Based Search Strategies for Constraint Programming. In *Proceedings of CP04*, pages 557–571, 2004.
19. W. Shi. A Branch and Bound Method for the Multiconstraint Zero One Knapsack Problem. *Journal of the Operational Research Society*, 30:369–378, 1979.
20. B. M. Smith and S. A. Grant. Trying Harder to Fail First. In *Thirteenth European Conference on Artificial Intelligence (ECAI 98)*, pages 249–253. John Wiley & Sons, 1998.
21. K. Stergiou. Heuristics for dynamically adapting propagation. In *ECAI2008*, pages 485–489, 2008.
22. A. Zanarini and G. Pesant. Solution Counting Algorithms for Constraint-Centered Search Heuristics. In *Constraints*, volume 14, pages 392–413, 2009.